

Skill Week 12

Aim

To execute a large Linux pipeline with multiple processes, monitor system resources, detect failures, and analyze the performance of the pipeline.

```
navaneeth@Navaneeth: ~/ski × + v
navaneeth@Navaneeth:~$ mkdir -p ~/skillweek12
navaneeth@Navaneeth:~$ cd ~/skillweek12
navaneeth@Navaneeth:~/skillweek12$ seq 1 10000 > numbers.txt
navaneeth@Navaneeth:~/skillweek12$ head numbers.txt
1
2
3
4
5
6
7
8
9
10
navaneeth@Navaneeth:~/skillweek12$ cat numbers.txt | grep "1" | sort -n | uniq | wc -l
3440
navaneeth@Navaneeth:~/skillweek12$ cat numbers.txt | grep "1" | sort -n | uniq > result.txt
navaneeth@Navaneeth:~/skillweek12$ sleep 30 &
sleep 40 &
sleep 50 &
[1] 2324
[2] 2330
[3] 2336
navaneeth@Navaneeth:~/skillweek12$ jobs -l
[1] 2324 Running sleep 30 &
[2]- 2330 Running sleep 40 &
[3]+ 2336 Running sleep 50 &
navaneeth@Navaneeth:~/skillweek12$ ps -o pid,ppid,stat,cmd
  PID  PPID  STAT  CMD
  2208   2205  Ss    -bash
  2324   2208  S      sleep 30
  2330   2208  S      sleep 40
  2336   2208  S      sleep 50
  2357   2208  R+    ps -o pid,ppid,stat,cmd
navaneeth@Navaneeth:~/skillweek12$ free -h
             total        used        free      shared  buff/cache   available
Mem:        7.6Gi        551Mi        6.9Gi        3.5Mi        308Mi        7.0Gi
Swap:        2.0Gi          0B          2.0Gi
[1] Done sleep 30
navaneeth@Navaneeth:~/skillweek12$ cat missing_file.txt
cat: missing_file.txt: No such file or directory
[2]- Done sleep 40
navaneeth@Navaneeth:~/skillweek12$ echo $?
1
navaneeth@Navaneeth:~/skillweek12$ time cat numbers.txt | grep "1" | sort -n | uniq | wc -l
3440

real    0m0.007s
user    0m0.014s
sys     0m0.008s
[3]+ Done sleep 50
navaneeth@Navaneeth:~/skillweek12$ jobs -l
navaneeth@Navaneeth:~/skillweek12$ kill %1
kill %2
kill %3
-bash: kill: %1: no such job
-bash: kill: %2: no such job
-bash: kill: %3: no such job
navaneeth@Navaneeth:~/skillweek12$ |
```

Observation

The Linux pipeline successfully processed the input data using multiple commands connected through pipes. Multiple background jobs were launched and monitored using jobs and ps. System resources were checked using free and uptime. A failed command was detected using its exit status, and pipeline execution time was measured using time.

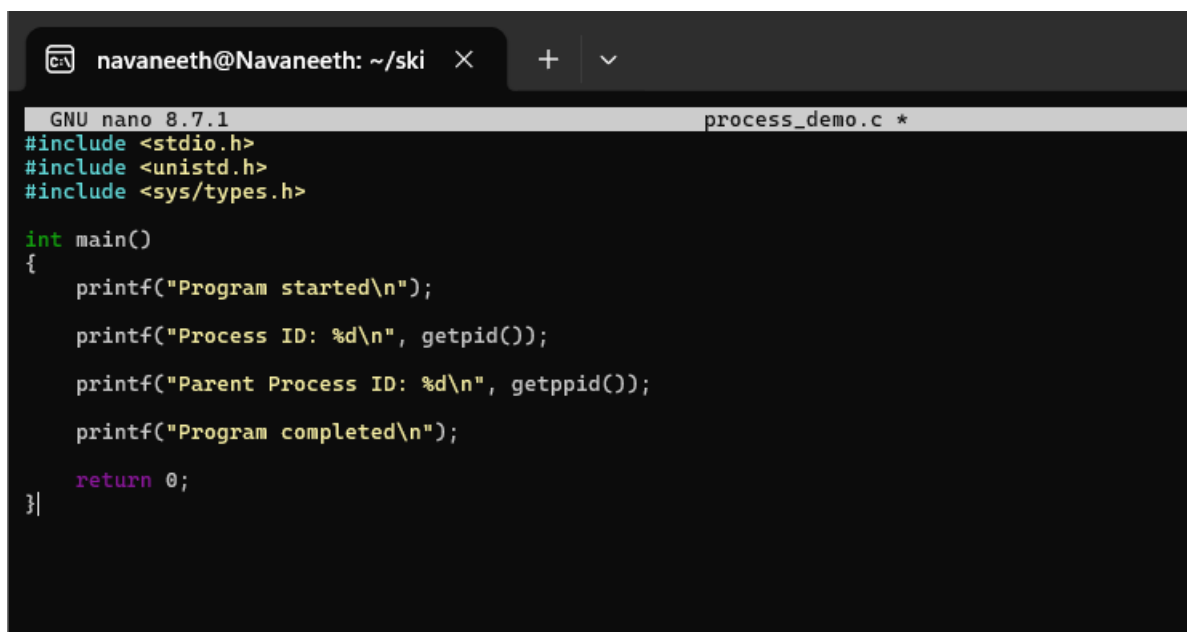
Result

Large pipelines and multiple jobs were executed successfully. Process monitoring, failure detection, resource monitoring, and performance analysis were performed.

Aim

To document the architecture of a Linux program, create an execution flow, identify system calls, and record important design decisions.

Program:



```
GNU nano 8.7.1 process_demo.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    printf("Program started\n");

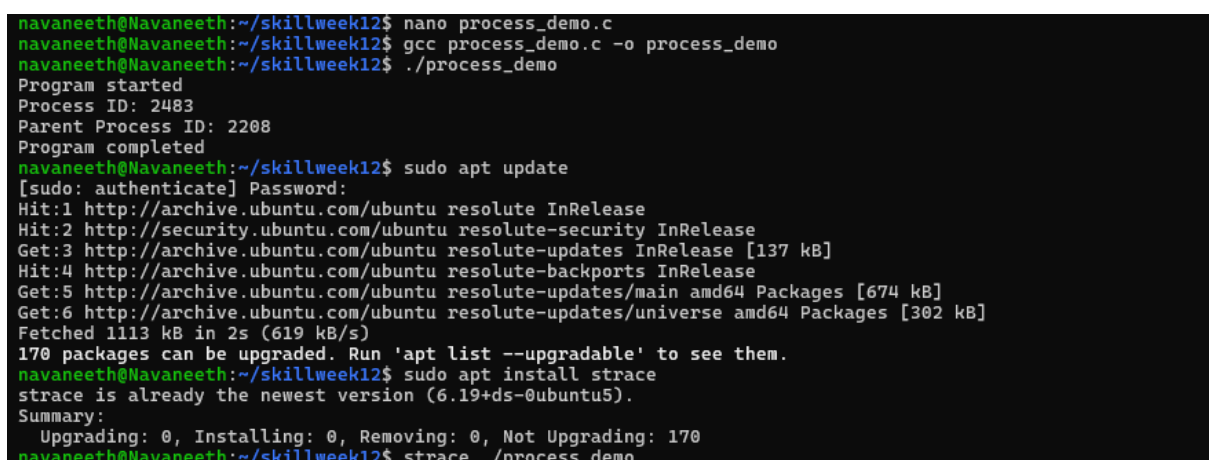
    printf("Process ID: %d\n", getpid());

    printf("Parent Process ID: %d\n", getppid());

    printf("Program completed\n");

    return 0;
}
```

Output:



```
navaneeth@Navaneeth:~/skillweek12$ nano process_demo.c
navaneeth@Navaneeth:~/skillweek12$ gcc process_demo.c -o process_demo
navaneeth@Navaneeth:~/skillweek12$ ./process_demo
Program started
Process ID: 2483
Parent Process ID: 2208
Program completed
navaneeth@Navaneeth:~/skillweek12$ sudo apt update
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://security.ubuntu.com/ubuntu resolute-security InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:5 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [674 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [302 kB]
Fetched 1113 kB in 2s (619 kB/s)
170 packages can be upgraded. Run 'apt list --upgradable' to see them.
navaneeth@Navaneeth:~/skillweek12$ sudo apt install strace
strace is already the newest version (6.19+ds-0ubuntu5).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 170
navaneeth@Navaneeth:~/skillweek12$ strace ./process_demo
```

```
navaneeth@Navaneeth: ~/ski x + v
navaneeth@Navaneeth:~/skillweek12$ nano architecture.txt
navaneeth@Navaneeth:~/skillweek12$ cat architecture.txt
LINUX PROGRAM ARCHITECTURE

1. User
|
v
2. Main Program
|
v
3. Process Management
|
v
4. Linux Kernel
|
v
5. System Calls
|
v
6. Output to Terminal

EXECUTION PATH

Program starts
|
v
main()
|
v
getpid()
|
v
getppid()
|
v
printf()
|
v
Program terminates

SYSTEM CALLS

execve() - Starts the program
write() - Writes output
getpid() - Gets process ID
getppid() - Gets parent process ID
exit_group() - Terminates the process

DESIGN DECISIONS

1. C language is used for Linux system-level programming.
2. Process IDs are displayed to understand process execution.
3. System calls are traced using strace.
4. The program is kept simple for easy testing and documentation.
5. Output is recorded for verification.
navaneeth@Navaneeth:~/skillweek12$ ls -l
total 96
-rw-r--r-- 1 navaneeth navaneeth 793 Sep 27 08:21 architecture.txt
-rw-r--r-- 1 navaneeth navaneeth 48894 Sep 27 08:14 numbers.txt
-rwxr-xr-x 1 navaneeth navaneeth 16088 Sep 27 08:19 process_demo
-rw-r--r-- 1 navaneeth navaneeth 256 Sep 27 08:19 process_demo.c
-rw-r--r-- 1 navaneeth navaneeth 16910 Sep 27 08:14 result.txt
-rw-r--r-- 1 navaneeth navaneeth 2658 Sep 27 08:20 syscall_log.txt
```

Observation

The program architecture and execution path were documented. The program displayed its process ID and parent process ID. System calls were traced using strace, and the system-call information was stored in syscall_log.txt.

Result

The Linux program architecture, execution path, system calls, and design decisions were successfully documented and reviewed.

Aim

To organize a Linux project source code, create a build script, prepare a README file, verify the project structure, and create a release tag

Program:

```
navaneeth@Navaneeth: ~/ski × + v
GNU nano 8.7.1 src/main.c *
#include <stdio.h>

int main()
{
    printf("Linux Project Build Successful\n");
    return 0;
}
```

```
navaneeth@Navaneeth: ~/ski × + v
GNU nano 8.7.1 Makefile *
CC = gcc
CFLAGS = -Wall

TARGET = project
SOURCE = src/main.c

all:
    $(CC) $(CFLAGS) $(SOURCE) -o $(TARGET)

clean:
    rm -f $(TARGET)
```

```
navaneeth@Navaneeth: ~/ski × + v
GNU nano 8.7.1 README.md *
# Linux Project

## Description
This project demonstrates basic Linux source-code organization
and automated compilation using a Makefile.

## Files
- src/main.c - Main C source file
- Makefile - Builds and cleans the project
- README.md - Project documentation

## Build
Run:

make

## Execute
Run:

./project

## Clean
Run:

make clean
```

Output:

```

navaneeth@Navaneeth:~/skillweek12$ mkdir -p ~/skillweek12/project
navaneeth@Navaneeth:~/skillweek12$ cd ~/skillweek12/project
navaneeth@Navaneeth:~/skillweek12/project$ mkdir src
navaneeth@Navaneeth:~/skillweek12/project$ nano src/main.c
navaneeth@Navaneeth:~/skillweek12/project$ nano Makefile
navaneeth@Navaneeth:~/skillweek12/project$ make
gcc -Wall src/main.c -o project
navaneeth@Navaneeth:~/skillweek12/project$ ./project
Linux Project Build Successful
navaneeth@Navaneeth:~/skillweek12/project$ make clean
rm -f project
navaneeth@Navaneeth:~/skillweek12/project$ ls
Makefile  src
navaneeth@Navaneeth:~/skillweek12/project$ make
gcc -Wall src/main.c -o project
navaneeth@Navaneeth:~/skillweek12/project$ nano README.md
navaneeth@Navaneeth:~/skillweek12/project$ find . -maxdepth 2 -type f
./README.md
./project
./Makefile
./src/main.c
navaneeth@Navaneeth:~/skillweek12/project$ git add .
fatal: not a git repository (or any of the parent directories): .git
navaneeth@Navaneeth:~/skillweek12/project$ git commit -m "Initial project release"
fatal: not a git repository (or any of the parent directories): .git
navaneeth@Navaneeth:~/skillweek12/project$ git tag v1.0

```

Observation

The project source code was organized into a separate src directory. A Makefile was created to automate compilation and cleaning. A README file documented the project. Git was initialized, the files were committed, and version tag v1.0 was created.

Result

The source code was organized, the build process was automated, documentation was added, the repository structure was verified, and the project was tagged as version 1.0.

Aim

To create and test different Linux demonstration scenarios, validate project features, demonstrate pipelines and signals, and perform a final project review.

Program:

```
navaneeth@Navaneeth: ~/ski X + v
GNU nano 8.7.1 signal_demo.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handle_signal(int sig)
{
    printf("\nSIGINT received. Program is still running.\n");
}

int main()
{
    signal(SIGINT, handle_signal);

    printf("Process started. Press Ctrl+C to send SIGINT.\n");

    while (1)
    {
        printf("Running...\n");
        sleep(2);
    }

    return 0;
}
```

Output:

```
navaneeth@Navaneeth: ~/ski X + v - □ X
navaneeth@Navaneeth:~/skillweek12/project$ cd ~/skillweek12
navaneeth@Navaneeth:~/skillweek12$ printf "apple\nbanana\napple\norange\nbanana\n" > fruits.txt
navaneeth@Navaneeth:~/skillweek12$ cat fruits.txt | sort | uniq
apple
banana
orange
navaneeth@Navaneeth:~/skillweek12$ cat fruits.txt | sort | uniq | wc -l
3
navaneeth@Navaneeth:~/skillweek12$ sleep 30 &
sleep 40 &
[1] 2993
[2] 2999
navaneeth@Navaneeth:~/skillweek12$ jobs -l
[1]- 2993 Running                  sleep 30 &
[2]+ 2999 Running                  sleep 40 &
navaneeth@Navaneeth:~/skillweek12$ ps -o pid,ppid,stat,cmd
  PID  PPID  STAT  CMD
  2208   2205  Ss    -bash
  2993   2208  S      sleep 30
  2999   2208  S      sleep 40
  3020   2208  R+     ps -o pid,ppid,stat,cmd
navaneeth@Navaneeth:~/skillweek12$ nano signal_demo.c
[1]- Done                          sleep 30
[2]+ Done                          sleep 40
navaneeth@Navaneeth:~/skillweek12$ gcc signal_demo.c -o signal_demo
navaneeth@Navaneeth:~/skillweek12$ ./signal_demo
Process started. Press Ctrl+C to send SIGINT.
Running...
Running...
^C
SIGINT received. Program is still running.
Running...
Running...
Running...
^Z
[1]+  Stopped                          ./signal_demo
navaneeth@Navaneeth:~/skillweek12$ jobs
[1]+  Stopped                          ./signal_demo
navaneeth@Navaneeth:~/skillweek12$ kill %1
[1]+  Stopped                          ./signal_demo
[1]+  Terminated                      ./signal_demo
```

```

navaneeth@Navaneeth:~/skillweek12$ find ~/skillweek12 -maxdepth 3 -type f
/home/navaneeth/skillweek12/fruits.txt
/home/navaneeth/skillweek12/process_demo
/home/navaneeth/skillweek12/signal_demo
/home/navaneeth/skillweek12/project/README.md
/home/navaneeth/skillweek12/project/project
/home/navaneeth/skillweek12/project/Makefile
/home/navaneeth/skillweek12/project/src/main.c
/home/navaneeth/skillweek12/architecture.txt
/home/navaneeth/skillweek12/signal_demo.c
/home/navaneeth/skillweek12/syscall_log.txt
/home/navaneeth/skillweek12/result.txt
/home/navaneeth/skillweek12/numbers.txt
/home/navaneeth/skillweek12/process_demo.c
navaneeth@Navaneeth:~/skillweek12$ ls -l ~/skillweek12/project
total 28
-rw-r--r-- 1 navaneeth navaneeth 133 Sep 27 08:25 Makefile
-rw-r--r-- 1 navaneeth navaneeth 338 Sep 27 08:27 README.md
-rwxr-xr-x 1 navaneeth navaneeth 15952 Sep 27 08:26 project
drwxr-xr-x 2 navaneeth navaneeth 4096 Sep 27 08:25 src
navaneeth@Navaneeth:~/skillweek12$ nano final_review.txt
navaneeth@Navaneeth:~/skillweek12$ cat final_review.txt
LINUX PROJECT FINAL REVIEW

```

1. Source Code
Source files were created and organized properly.
2. Build Process
The Makefile was used to compile the project.
3. Pipeline Demonstration
Multiple Linux commands were connected using pipes.
4. Process Management
Background processes were created and monitored.
5. Signal Demonstration
SIGINT was captured and handled by the C program.
6. Testing
Program outputs and important features were verified.
7. Documentation
README and project documentation were prepared.
8. Final Validation
Project files, compilation, execution and demonstrations were reviewed successfully.

```

navaneeth@Navaneeth:~/skillweek12$ |

```

Observation

The project features were demonstrated and validated. Linux pipelines were executed, background processes were monitored, and signal handling was demonstrated using a C program. The project files and documentation were reviewed before the final presentation.

Result

Demo scenarios, pipeline execution, signal handling, feature validation, presentation rehearsal, and final project review were completed successfully.